

Resource-Anomaly Detector for O-RAN xApps

MEWMA / Hotelling- T^2 / Ledoit-Wolf Covariance — Model v5.0

Why MEWMA ?

MEWMA (multivariate EWMA) : Multivariate Exponentially Weighted Moving Average

- **MEWMA over a simple moving average**: Gives an **exponentially-decaying** memory of **recent behavior** without a hard window , so a moderate, sustained deviation accumulates evidence smoothly rather than needing a single dramatic spike
- **Multivariate**: a single scalar (T^2) fuses CPU, memory-growth, RX and TX while accounting for how they normally co-vary

$$Z_t = \lambda \cdot x_t + (1 - \lambda) \cdot Z_{t-1}$$

Where:

- X_t = Current metric vector

Example: [CPU, Memory, Network_RX, Network_TX, ... etc]

- Z_t = Smoothed MEWMA value at time t
- Z_{t-1} = Previous smoothed value
- λ = Smoothing factor [Chosen from latency vs FP / Responsiveness against noise]

The 11 collected metrics and which 4 feed the score.....

ID	Column	Units	Meaning	Role in model
M1	m1_cpu	millicores	CPU usage	SCORE (log1p) + rate gate
M2	m2_cpu_throttle_ratio	millicores	CFS throttle ratio	Quality guard only
M3	m3_memory_working_set	Bytes	Working-set memory	REMOVED in v2 (caused FPs)
M4	m4_memory_growth	Bytes/s	Memory growth rate	SCORE (identity)
M5	m5_network_rx	Bytes/s	Network RX rate	SCORE (log1p)
M6	m6_network_tx	Bytes/s	Network TX rate	SCORE (log1p)
M7	m7_filesystem_read	Bytes/s	FS read rate	Deterministic guard
M8	m8_filesystem_write	Bytes/s	FS write rate	Sparse-event guard
M9-M11	socket/fd/thread counts	-	Process resource counts	Invariant guards

Only 4 of 11 enter the score model. The precision matrix is 4x4 [M1, M4, M5, M6], [M3]REMOVED in v2 (caused FPs) and Rest chosen for the data quality

Ledoit-Wolf shrinkage covariance : (Checking each metric against its own separate can miss a genuinely attack)

- Technique for estimating a covariance matrix when features are highly correlated (Also T^2 needs the INVERSE covariance)

A raw sample covariance can be numerically fragile to invert if sampling noise can occasionally produce a wildly unstable inverse (M1 and M4 which makes the inverse numerically unstable)

- The Ledoit-Wolf shrinkage value is a number the algorithm **computes automatically** to minimize expected estimation error.

shrinkage coefficient = 0.0002496894

(small — the sample covariance was already fairly well-conditioned; shrinkage is a safety margin, not a rescue).

[1]

Symbol	Name	What it is
n	sample size	Number of observations. (pooled N1+N2 post-warmup MEWMA states)
p	dimensionality	Number of features. Here: 4 (M1, M4, M5, M6)
x_i	one observation	The i'th sample, a p-dimensional vector (one MEWMA state)
S	sample covariance	The matrix <i>S</i> raw covariance estimate.
I	identity matrix	The simplest, always-safely-invertible covariance shape imaginable.
Trace(S), Tr(S)	trace	Sum of S's diagonal entries — i.e., sum of each individual feature's own variance. A single number.
μ	shrinkage-target scale	$\mu = \text{Tr}(S)/p$ — the average variance across all p features. Just Trace divided by dimension count.
F	shrinkage target	$F = \mu \cdot I$ — a matrix with μ on every diagonal entry and 0 off-diagonal. This is the "safe fallback" structure: assumes all 4 features have the same variance and zero correlation. Always invertible, no matter what.
δ^2	distance from target	$\delta^2 = \ S - F\ _F^2$ (squared Frobenius norm — sum of squared entrywise differences). Measures how much real structure <i>S</i> has that differs from the naive "everything independent, same variance" assumption.
β^2 (beta-bar-squared)	sampling noise in <i>S</i>	For each individual sample, look at its own single-sample covariance contribution and measure how far that one sample deviates from the overall average <i>S</i> . Average this over all n samples. Large β^2 means <i>S</i> itself is noisy — draw a different sample and get a meaningfully different <i>S</i> .
β^2 (beta-squared)	capped noise term	$\beta^2 = \min(\beta^2, \delta^2)$ — a technical safeguard so the shrinkage fraction below never exceeds 1 (100%).
α^*	optimal shrinkage intensity	$\alpha^* = \beta^2 / \delta^2$ — the actual number is 0.0002496894. The fraction of the way to shrink <i>S</i> toward <i>F</i> .
Σ_{shrunk}	final output	$\alpha^* \cdot F + (1 - \alpha^*) \cdot S$ — the matrix actually used (before inverting it to get the precision matrix).

X shape: n = 3490 rows (timesteps) x 4 columns (p) (M1,M4,M5,M6)

First 5 rows of X (real MEWMA states):

row 0: [0.7015 0.0611 21.942 15.4992]
row 1: [0.7353 0.0485 23.586 16.7787]
row 2: [0.7339 0.0385 24.8907 17.7941]
row 3: [0.7059 0.0306 23.9816 17.4509]
row 4: [0.602 0.0243 21.2149 15.5067]

(all the 3490 more rows)

$S[0,0]$ (M1 variance) = $(1/n) * \text{sum of } (M1 \text{ value} * M1 \text{ value})$
across all rows

sum of all row products = 2273.5433

$S[0,0] = 2273.5433 / 3490 = \mathbf{0.6514}$

$S[0,1]$ (M1, M4 covariance) = $(1/n) * \text{sum of } (M1_t * M4_t)$ across
all rows

sum of all row products = 1027.9432

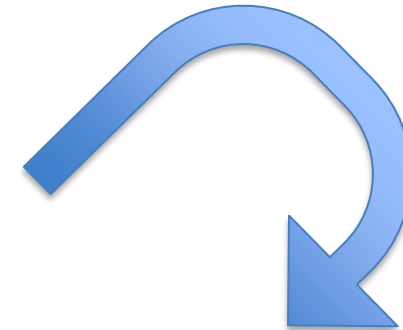
$S[0,1] = 1027.9432 / 3490 = \mathbf{0.2945}$

30*2*4

13960

9093.544

4111.22



Full S via matrix formula

[0.6514	0.2945	16.3256	10.3243]
[0.2945	0.3954	8.6105	5.5041]
[16.3256	8.6105	512.3698	312.9993]
[10.3243	5.5041	312.9993	197.8463]

$n = 3490, p = 4$

S (raw sample covariance, 4x4):

```
[ 0.6514  0.2945 16.3256 10.3243 ]
[ 0.2945  0.3954  8.6105  5.5041  ]
[16.3256  8.6105 512.3698 312.9993]
[10.3243  5.5041 312.9993 197.8463]
```

Trace : $\text{Tr}(S) = 711.262970$

shrinkage-target scale:

$\mu = \text{Tr}(S)/p = 711.262970/4 = 177.815742$

$F = \mu * I$ (shrinkage target):

```
[177.8157  0.    0.    0.  ]
[ 0.   177.8157  0.    0.  ]
[ 0.    0.   177.8157  0.  ]
[ 0.    0.    0.   177.8157]
```

$S - F$ (the difference matrix):

```
[-177.1643  0.2945 16.3256 10.3243 ]
[ 0.2945 -177.4204  8.6105  5.5041  ]
[ 16.3256  8.6105 334.5541 312.9993]
[10.3243  5.5041 312.9993 20.0306 ]
```

$\delta^2 = \text{sum of squared entries of } (S-F) = \mathbf{372085.295946}$

Sampling noise in S :

$$\beta^2 = \frac{1}{n^2} \sum_{i=1}^n \|x_i x_i^T - S\|_F^2 = \mathbf{92.905751}$$

x_i — the i -th observation vector (one MEWMA state, at one time step)

$x_i x_i^T$ — the outer product of that observation with itself, producing a small matrix

S — the overall sample covariance matrix (the average of all the $x_i x_i^T$ terms)

$x_i x_i^T - S$ — how far this one observation's contribution deviates from the overall average

$\|\cdot\|_F^2$ — the squared Frobenius norm: square every entry of that difference matrix and sum them all up, giving a single number representing "how big" the deviation is

$\sum_{i=1}^n x x x x$ — add that squared deviation up across all n observations

$\frac{1}{n^2}$ — normalize by the sample size squared..

$$\beta^2 = \min(\beta^2, \delta^2) = \min(92.905751, 372085.295946) = 92.905751$$

$$\alpha^* = \beta^2 / \delta^2 = 92.905751 / 372085.295946 = 0.0002496894$$

Sigma_shrunk = $\alpha^* \times F + (1 - \alpha^*) \times S$
(the final covariance, 4x4):

```
[ 0.6957  0.2945 16.3216 10.3217 ]  
[ 0.2945  0.4397  8.6083  5.5027 ]  
[16.3216  8.6083 512.2863 312.9212 ]  
[10.3217  5.5027 312.9212 197.8413 ]
```

$$\text{sklearn LedoitWolf().shrinkage} = 0.0002496894$$

($\alpha^* = 0.0002496894$)

Precision matrix = inverse(Sigma_shrunk):

```
[ 6.3699 -0.1638 0.0006 -0.3287]  
[-0.1638  3.4997 0.0192 -0.1191]  
[ 0.0006  0.0192 0.0578 -0.0919]  
[-0.3287 -0.1191 -0.0919  0.1709]
```

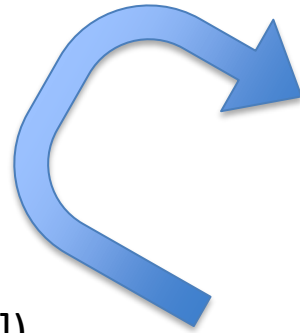
Every off-diagonal value, worked out from your S matrix

$$\text{corr}(M1, M4) = S[0,1] / \sqrt{S[0,0] * S[1,1]}$$

$$= 0.2945 / \sqrt{0.6514 * 0.3954}$$

$$= 0.2945 / \sqrt{0.2576}$$

$$= 0.2945 / 0.5075 = 0.5803$$



Feature Correlation Metrics

	M1(CPU)	M4(memgrow)	M5(RX)	M6(TX)
M1(CPU)	1.0000	0.5803	0.8936	0.9094
M4(memgrow)	0.5803	1.0000	0.6049	0.6223
M5(RX)	0.8936	0.6049	1.0000	0.9831
M6(TX)	0.9094	0.6223	0.9831	1.0000

$$\text{corr}(M1, M5) = S[0,2] / \sqrt{S[0,0] * S[2,2]}$$

$$= 16.3256 / \sqrt{0.6514 * 512.3698}$$

$$= 16.3256 / \sqrt{333.7577}$$

$$= 16.3256 / 18.2690 = 0.8936$$

$$\text{corr}(M1, M6) = S[0,3] / \sqrt{S[0,0] * S[3,3]}$$

$$= 10.3243 / \sqrt{0.6514 * 197.8463}$$

$$= 10.3243 / \sqrt{128.8771}$$

$$= 10.3243 / 11.3524 = 0.9094$$

Hotelling's T^2 is *mathematically defined* to need the **covariance matrix** specifically, because it has to combine two separate pieces of information

a) how features correlate with each other, and **(b)** how much each one naturally varies on its own. A correlation matrix only keeps (a) — by construction, it normalizes every feature to unit variance (b) entirely
In here (b) is *not* trivial information to discard — M5/M6 have variance 512/198 in the smoothed state space while M1/M4 have variance 0.65/0.40, a 500–1000× difference. (**From S matrix**)

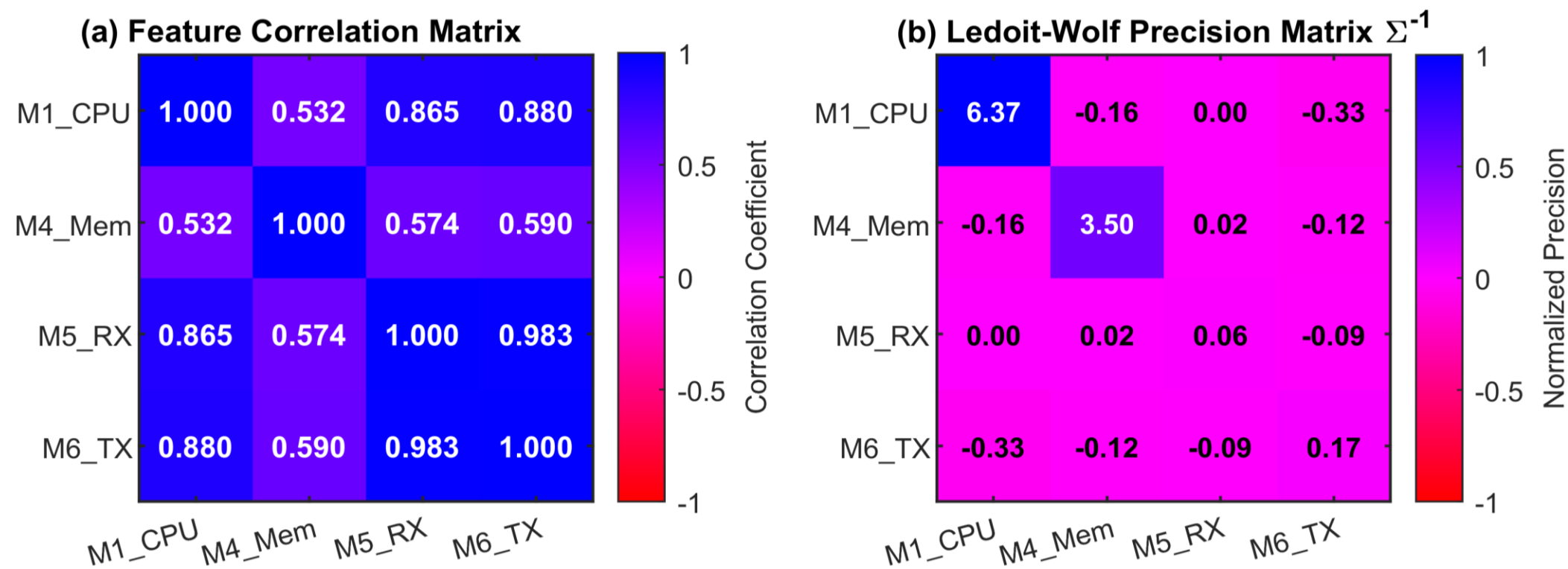
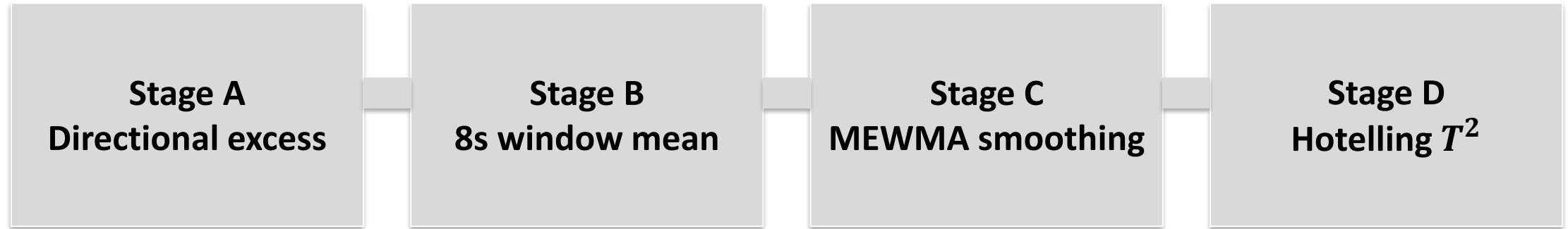


Figure : Feature Correlation vs Ledoit-Wolf Precision Metrix

This mismatches a "unit-variance-assuming" matrix against a state vector that was never actually rescaled to unit variance, producing a miscalibrated number that would systematically under- or over-weight different features

The 4-stage pipeline



Each stage takes the PREVIOUS stage's output as input

raw metric (11)

- [A] one 4-vector per sample (rectified, standardized)
- [B] one 4-vector per second (8s rolling mean)
- [C] one 4-vector per second (exponentially smoothed)
- [D] one scalar per second (T^2)

Stage A — directional excess all 4 features

transformed = log1p(raw) [M1, M5, M6] OR transformed = raw [M4, identity]

standardized = (transformed - center) / effective_scale

excess = max(0, standardized) **## one-sided rectification: only ABOVE-baseline deviation counts**

Ex:

Feature	raw (t=635.004s)	transform	center	eff. scale	standardized	excess
M1 (CPU)	1.056 mCPU	log1p->0.7208	0.85229	0.14962	-0.8791	0 (below baseline)
M4 (mem growth)	493759.4 B/s	identity	906.57	3322.79	148.325	148.325
M5 (RX)	536.581 B/s	log1p->6.2871	6.51111	0.00767	-29.211	0
M6 (TX)	838.783 B/s	log1p->6.7331	6.88395	0.00767	-19.663	0

Why M1 excess = 0 here: at t=635s the CPU is still BELOW its frozen baseline center (pre-attack). The rectification max(0,.) discards below-baseline deviation entirely — this is the v2->v3 fix (Point 1).

Stage B — 8-second sliding-window mean

$$x_t = (1/8) * \text{SUM}(\text{excess}[t-7 \dots t]) \quad (\text{arithmetic mean, stride 1 second})$$

Real 11-row excess sequence from R1_FRESH (columns: M1, M4, M5, M6):

Ex:

t (s)	M1 excess	M4 excess	M5 excess	M6 excess
635.004	0.0000	148.3251	0.0000	0.0000
636.002	0.0000	140.0996	0.0000	0.0000
637.003	0.0000	67.7692	0.0000	0.0000
638.002	2.6976	64.1169	51.7391	38.0660
639.002	0.0000	22.8044	0.0000	0.0000
640.003	0.0000	19.6975	0.0000	0.0000
641.003	19.2213	17.6107	0.0000	0.0000
642.002 (window 1 ends)	29.7683	16.8784	0.0000	0.0000

First complete window mean (rows 1-8, t=635.0-642.0s):

This is x_t , the vector Stage C (MEWMA) actually consumes — NOT the raw per-second excess.

mean(M1) = 6.4609
mean(M4) = 62.1627
mean(M5) = 6.4674
mean(M6) = 4.7583

Why a window mean at all? It suppresses single-sample noise BEFORE the exponential smoothing gets a chance to over-react to one outlier sample.

Stage C — MEWMA, and what λ actually means

$$Z_t = \lambda \cdot x_t + (1 - \lambda) \cdot Z_{t-1} \quad (\text{reset every run})$$

What IS lambda, conceptually?

- λ is a RECENCY-WEIGHTING knob, separate from the window size.
- λ decides how much of the smoothed HISTORY to keep vs. how much weight the CURRENT x_t gets.
- λ near 1 $\rightarrow Z_t$ almost equals x_t (short memory, reacts fast, noisier).
- λ near 0 $\rightarrow Z_t$ barely moves from Z_{t-1} (long memory, very smooth, slow to react)
- The half-life H is the more intuitive fact: 'after H seconds, a past window-mean's influence has decayed to 50%'. $\lambda = 1 - 2^{\frac{-1}{H}}$ converts that intuitive H into the recursion weight.

Division of labour: window size (Stage B) suppresses per-second noise; lambda (Stage C) controls how long evidence persists once it enters the smoothed state.

Worked recursion, real numbers (lambda=0.206299, $Z_0 = 0$)

$$\begin{aligned} z_1 &= 0.206299 \cdot [6.461, 62.163, 6.467, 4.758] + 0.793701 \cdot [0, 0, 0, 0] \\ &= [1.333, 12.824, 1.334, 0.982] \end{aligned}$$

$$z_2 = 0.206299 \cdot x_2 + 0.793701 \cdot z_1$$

$$z_3 = 0.206299 \cdot x_3 + 0.793701 \cdot z_2$$

$$\begin{aligned} z_4 &= 0.206299 \cdot x_4 + 0.793701 \cdot z_3 \\ &= [7.723, 22.039, 7.111, 4.602] \end{aligned}$$

*****final state used for T2

z accumulates: even though each x_t is a fresh window-mean, z_4 carries forward roughly 79% of z_3 's evidence at every step, which is why sustained (not just momentary) pressure is what drives T2 up.

The FULL candidate window x half-life grid

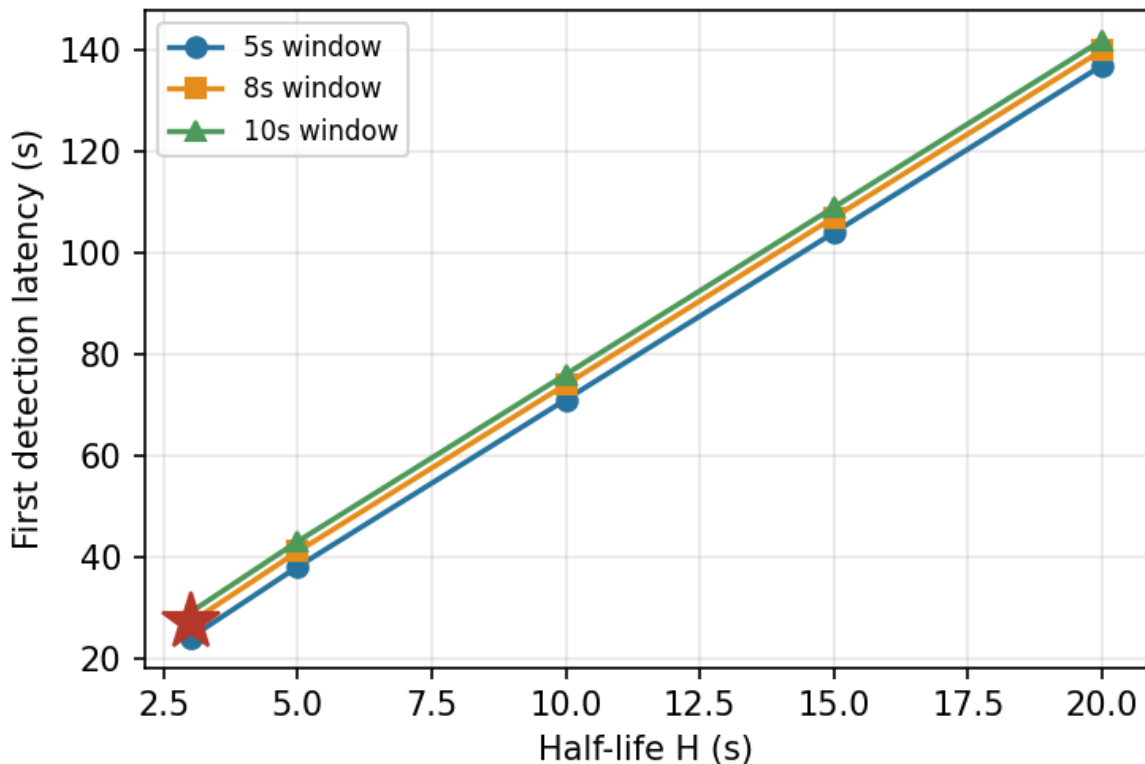
Window	Half-life H	λ	Warmup	1st score latency	N3 p50	N3 p90	N3 p99	N3 max
5s	3s	0.206299	20	27s	208.7	259.9	320.3	332.6
5s	5s	0.129449	34	38s	243.0	302.1	371.8	384.6
5s	10s	0.066967	67	71s	309.0	382.3	457.7	462.1
5s	15s	0.045158	100	104s	375.1	460.3	537.8	545.1
5s	20s	0.034064	133	137s	468.7	571.5	630.2	662.6
8s *FROZEN*	3s	0.206299	20	24s	222.3	277.0	341.8	350.0
8s	5s	0.129449	34	41s	252.3	313.6	386.0	397.9
8s	10s	0.066967	67	74s	315.3	389.9	466.6	471.5
8s	15s	0.045158	100	107s	382.0	468.2	546.2	553.8
8s	20s	0.034064	133	140s	477.3	582.4	641.0	670.8
10s	3s	0.206299	20	29s	230.2	287.6	353.9	361.9
10s	5s	0.129449	34	43s	258.2	320.9	394.7	406.3
10s	10s	0.066967	67	76s	319.6	394.6	472.6	476.7
10s	15s	0.045158	100	109s	386.8	474.2	552.0	560.3
10s	20s	0.034064	133	142s	483.2	588.7	649.5	676.8

*The frozen choice (8s, H=3s) is row 6, marked *FROZEN*. All 15 rows evaluated identically on N3 before any attack data existed.*

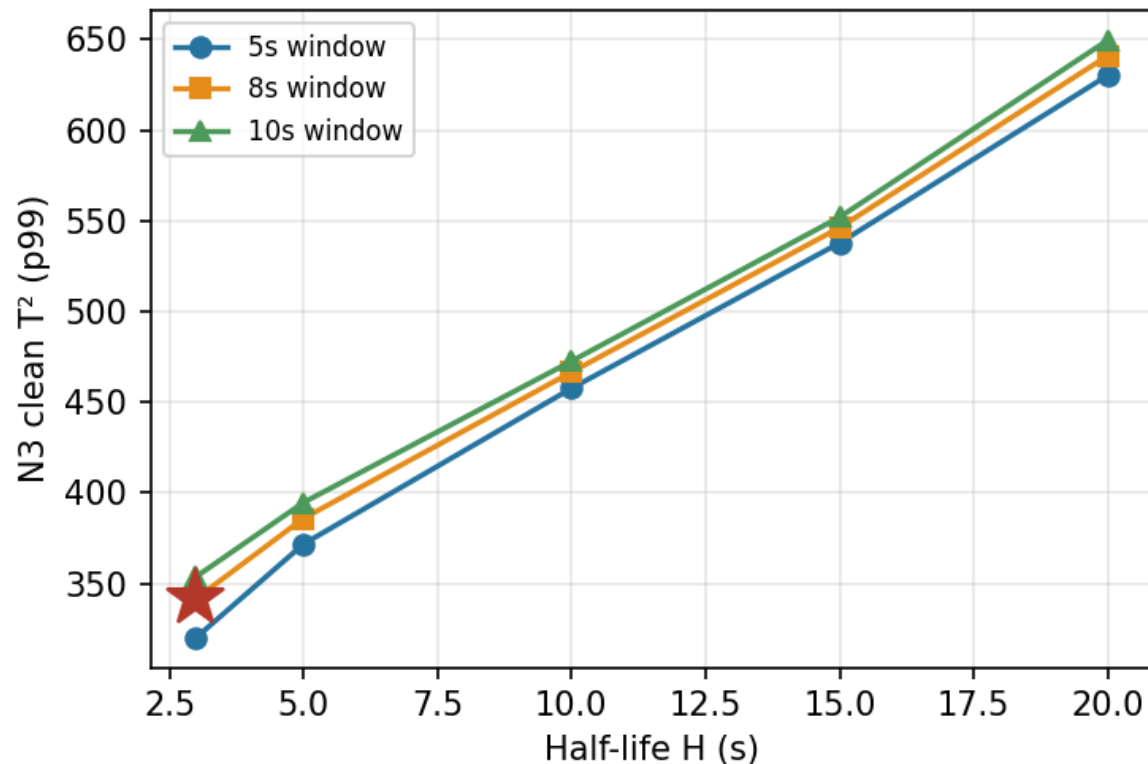
The latency/noise tradeoff, and the honesty flag

PREVIEW - camera-ready version: fig_window_lambda_selection.m

(a) Latency vs Half-Life



(b) Clean-Score Spread vs Half-Life



```
latency_floor      = window_seconds - 1
mean_evidence_age = (window_seconds-1)/2
```

Tradeoff formulas (in code, windows.py)

- Larger window -> smoother mean, more robust to ONE legitimate high-workload spike -> but LATER detection.
- 5s->27s latency, 8s->24s latency, 10s->29s latency (H=3s row). The frozen 8s trades 3s vs. 5s for a smoother window.

Stage D — Hotelling T², scaling, and the full arithmetic

$$T_t^2 = Z_t^T \times \text{inverse}(\text{Sigma_shrunk}) \times Z_t$$

T² is the "joint check": it learns how CPU, memory-growth, RX, and TX normally move together (the covariance matrix), then measures how far current behaviour sits from that *joint* relationship — not from four separate numbers.

Where does 'scaling' actually happen?

TWO separate scalings, at two different stages — neither is inside the T² formula itself:

(1) Stage A per-feature scaling: dividing by effective_scale converts CPU (mCPU), memory-growth (bytes/s), and RX/TX (log-bytes/s) onto comparable, unitless axes BEFORE they ever reach the covariance matrix.

(2) Implicit whitening inside Sigma_inv: the precision matrix's diagonal entries are roughly 1/variance per feature (6.37, 3.50, 0.058, 0.171) — features with small natural variance in state-space (like M5) get amplified relative to features with large variance (like M1), so T² does not let one feature dominate just because its raw numbers are bigger.

Precision matrix = inverse(Sigma_shrunk):

```
[ 6.3699 -0.1638 0.0006 -0.3287]
[-0.1638 3.4997 0.0192 -0.1191]
[ 0.0006 0.0192 0.0578 -0.0919]
[-0.3287 -0.1191 -0.0919 0.1709]
```

The actual T2 arithmetic, step by step

```
Step 1: Sigma_inv @ z_4 (matrix x vector)
row1: 6.37*7.723 -0.164*22.04 +0.0006*7.11
      -0.329*4.60 = 44.077
row2: -0.164*7.72+3.50*22.04+0.019*7.11
      -0.119*4.60 = 75.454
row3: 0.0006*7.72+0.019*22.04+0.058*7.11
      -0.092*4.60 = 0.415
row4: -0.329*7.72-0.119*22.04-0.092*7.11
      +0.171*4.60 = -5.031
```

4x1 metric

Step 2: dot with z_4 itself

```
T^2 = 7.723*44.077 + 22.039*75.454 + 7.111*0.415 + 4.602*(-5.031)
     = 340.41+1663.0+2.95-23.16 = 1983.17
```

1x1 metric

Compare to WL=10.4364, UCL=15.3084 -> 1983.17 is ~130x the UCL: an unambiguous, deep attack signal (this window sits well inside the attack phase).

```
z_4 = 0.206299*x_4 + 0.793701*z_3
     = [7.723, 22.039, 7.111, 4.602]
```

Other equations — persistence and gates

Mechanism	Equation	Frozen value	Why
6-of-8 persistence	<code>count(T2>=UCL, last 8) >= 6</code>	confirm rule	single-sample spikes don't fire alone
CPU rate gate	<code>frac(CPU>floor, last 60s) > 0.15</code>	floor=56.638 mCPU	duty-cycling attacker can't hide
MEWMA warmup	<code>ceil(log(0.01)/log(1-lam))</code>	20 states @ lam=0.2063	discard transient before z stabilizes

Why a RATE gate, not a MEAN gate (the duty-cycle problem)

- An earlier version (v4.0) used a windowed MEAN of CPU as the corroborating signal. Problem: an attacker busy 50% of the time and idle 50% of the time has a MEAN that looks only moderately elevated — easily confused with legitimate bursty load.
- The rate-of-exceedance gate instead counts the FRACTION OF SAMPLES that individually cross the floor. A duty-cycling attacker still crosses the floor every time it's 'on', so its rate stays high even though its mean looks moderate.

This was the specific, documented motivation for the Step 76 (v5.0) redesign — not a hypothetical concern.

Threshold derivations — Wilks bound and empirical quantiles

CPU floor: Wilks (1941) tolerance bound

- NOT a percentile of T2. A one-sided NONPARAMETRIC tolerance bound: with 95% confidence, at least 99.9% of FUTURE clean CPU samples fall below it.
- Computed purely from the empirical ORDER STATISTICS of pooled N1+N2+N3 CPU samples (n=5400) — no Gaussian or chi-square assumption needed.

Result: floor = 56.638 mCPU.

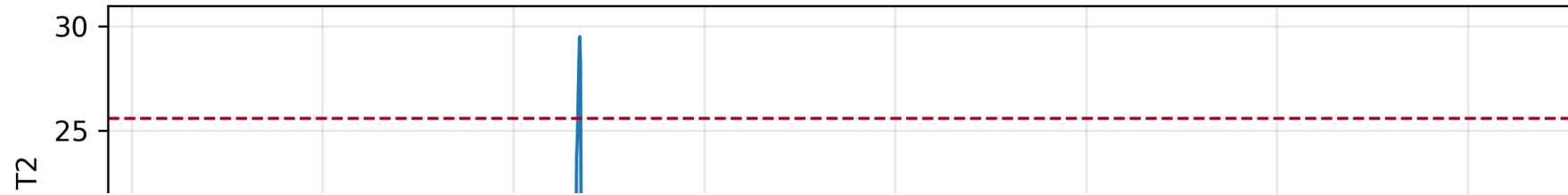
Warning limit & UCL: empirical N3 quantiles

```
warning_limit = quantile(N3 T2 scores, 0.975) = 10.4364  
UCL           = quantile(N3 T2 scores, 0.995) = 15.3084
```

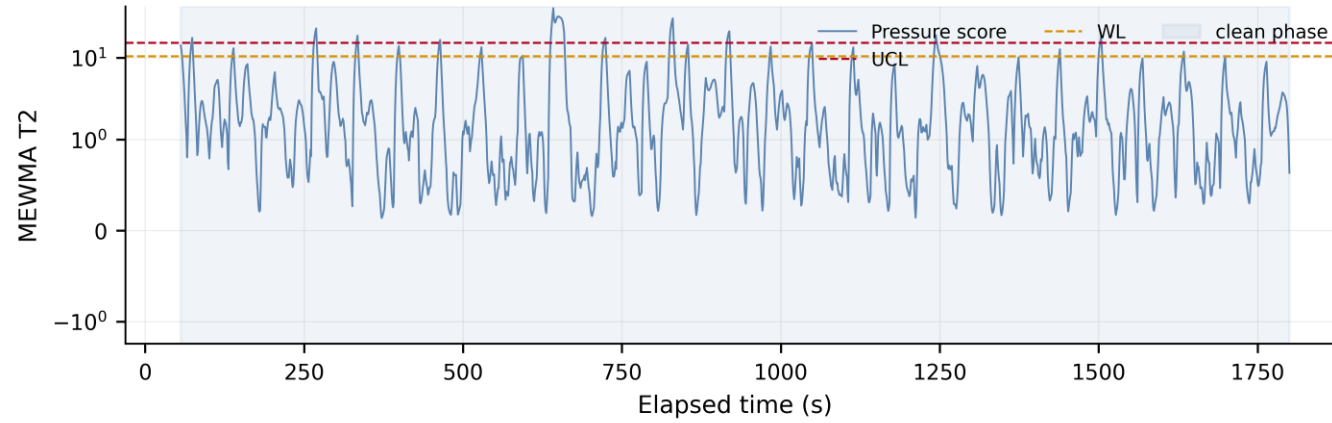
CONSTRUCTION (max(0,.) truncates the left tail entirely). A classical chi-square limit assumes multivariate normality — wrong model here.

- Empirical quantiles of REAL clean N3 scores sidestep that assumption completely — the threshold is exactly calibrated to this detector's actual score distribution, whatever shape it has.

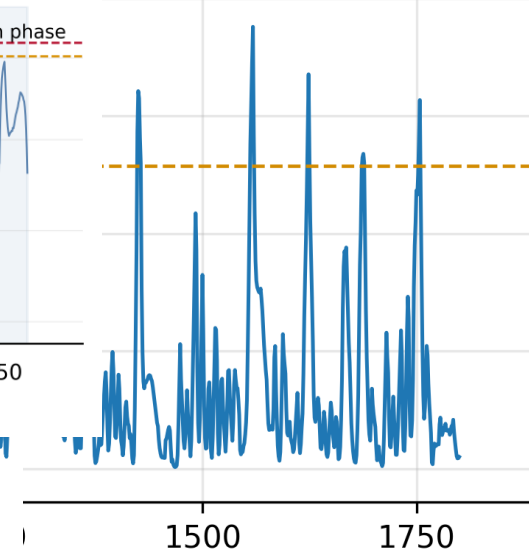
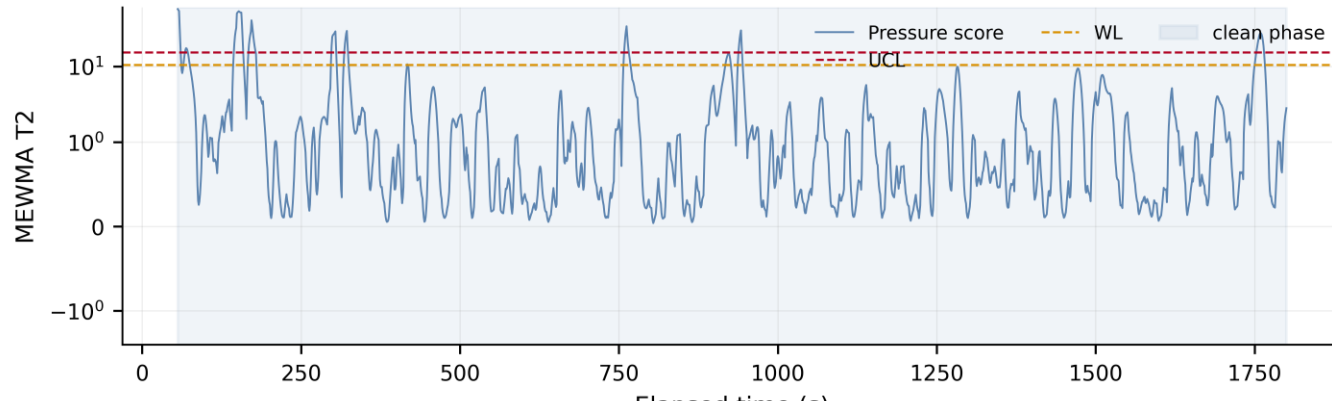
Step 45 Model v2 N4 Post-Hoc Score Timeline



N5 (clean, held-out)



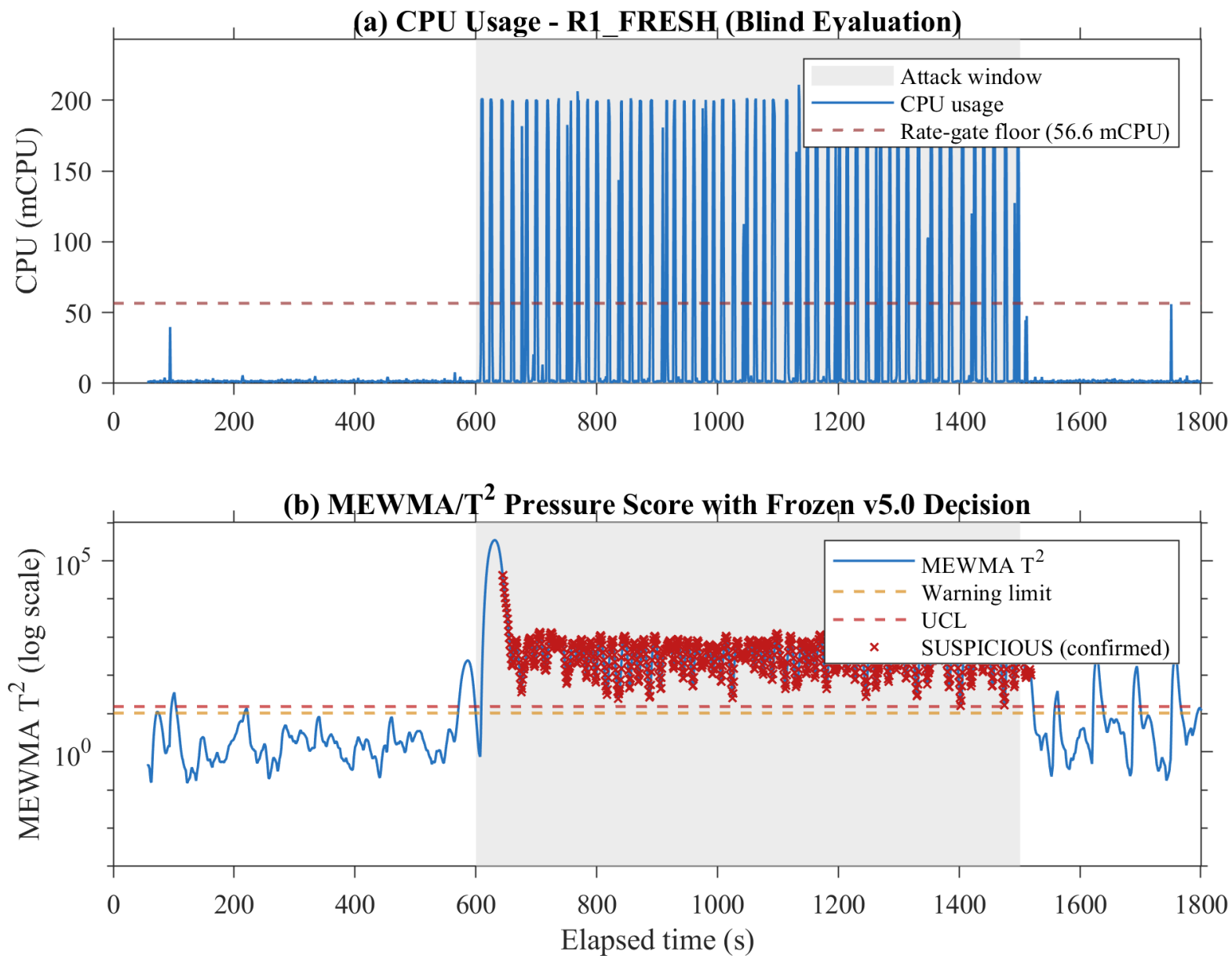
D4 (clean, held-out)



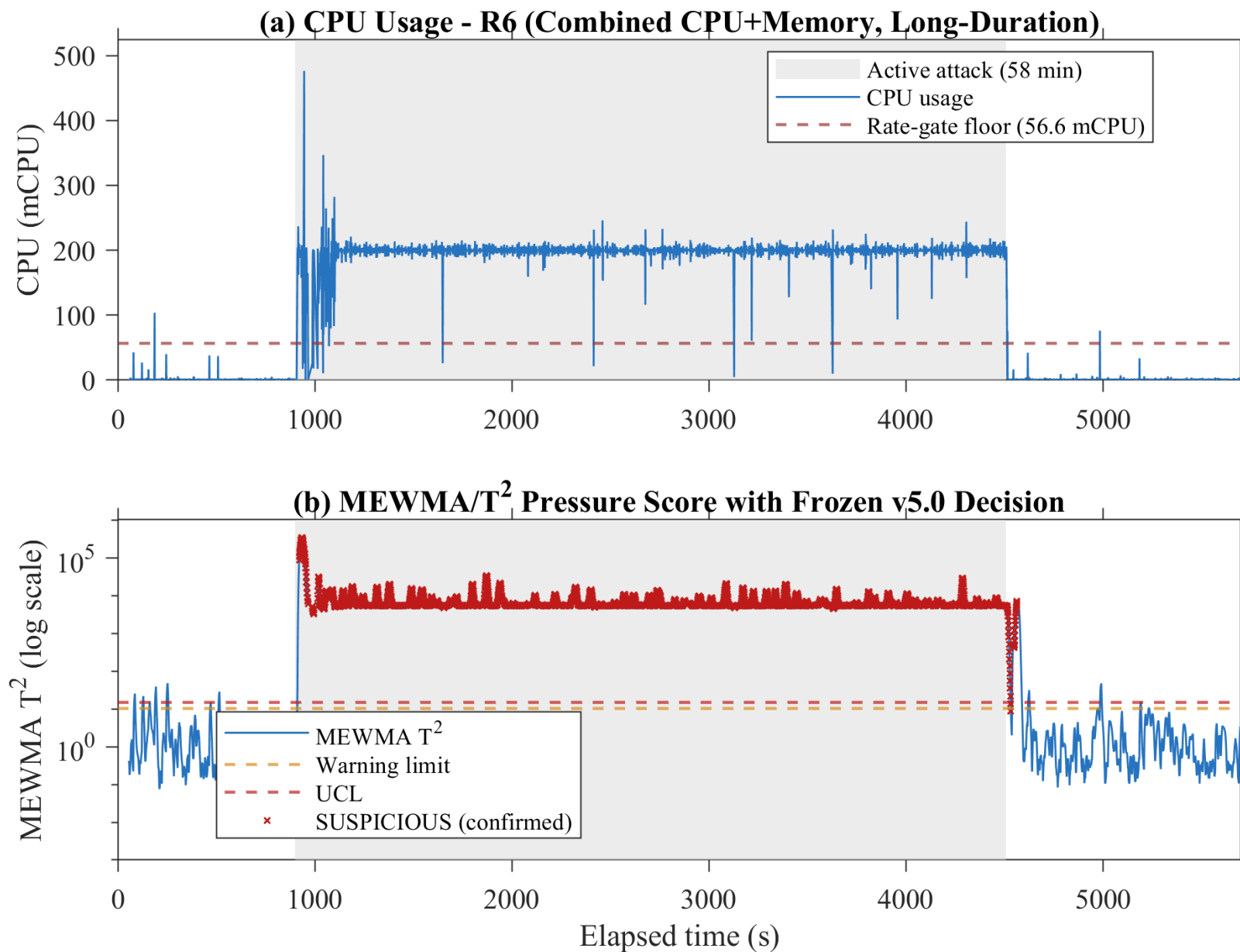
Evaluation Scenarios: R1, R6, A5, A_Memory, A_Burst, A_Stealth

Scenario	Adversary strategy	Attack?
R1	Brute-force sustained CPU exhaustion	Yes
R6	Combined CPU + memory	Yes
A5	Legitimate elevated load (no attack)	No — benign control
A_Memory	Attacks memory	Yes
A_Burst	Intermittent short bursts (5-60s)	Yes
A_Stealth	Low-and-slow near the detection floor	Yes

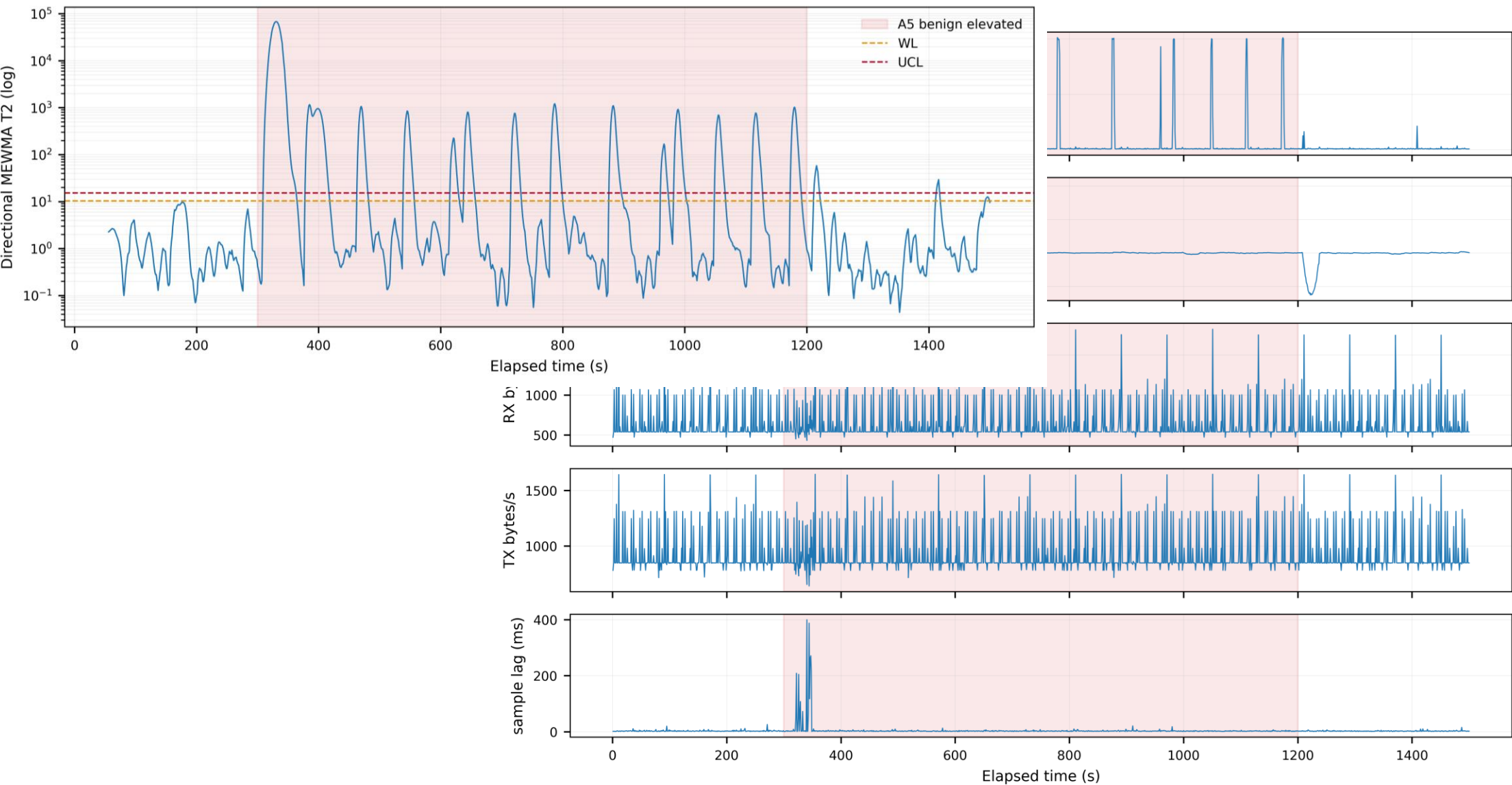
Evaluation Using R1 Dataset (Sustained CPU Exhaustion)



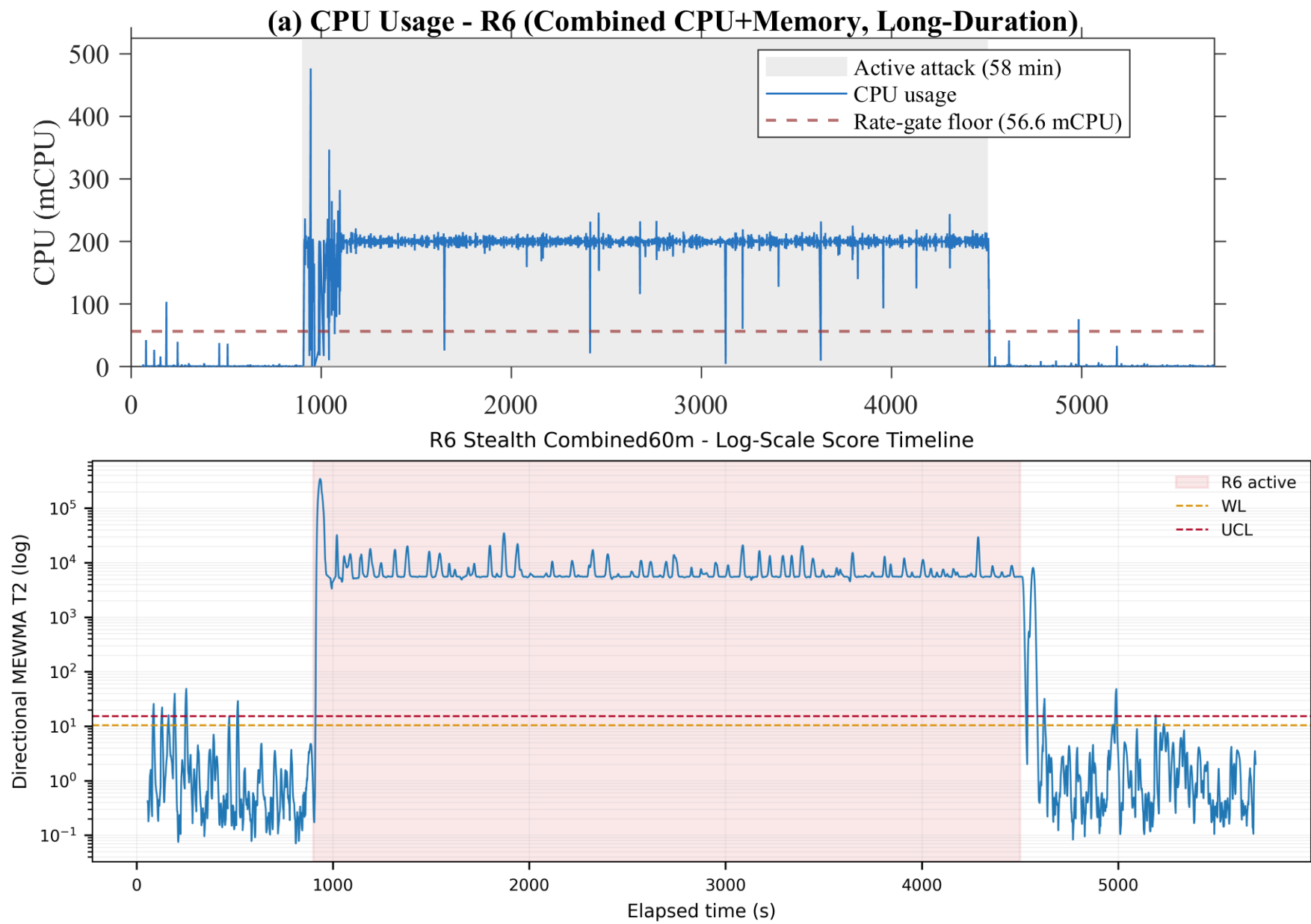
Evaluation Using R6 Dataset (CPU + Memory Combined)



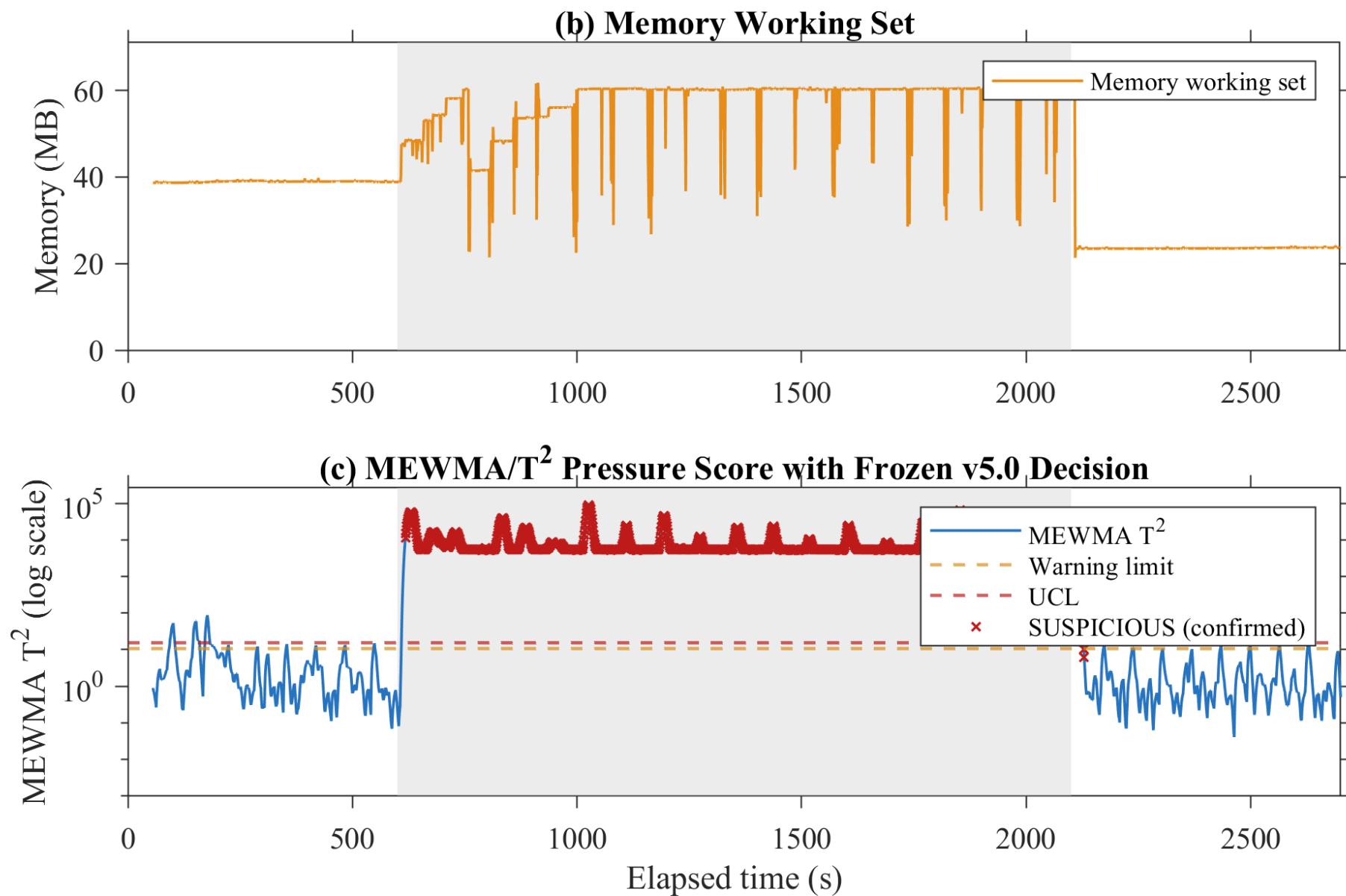
Evaluation Using A5 Dataset (Legitimate Elevated Load)



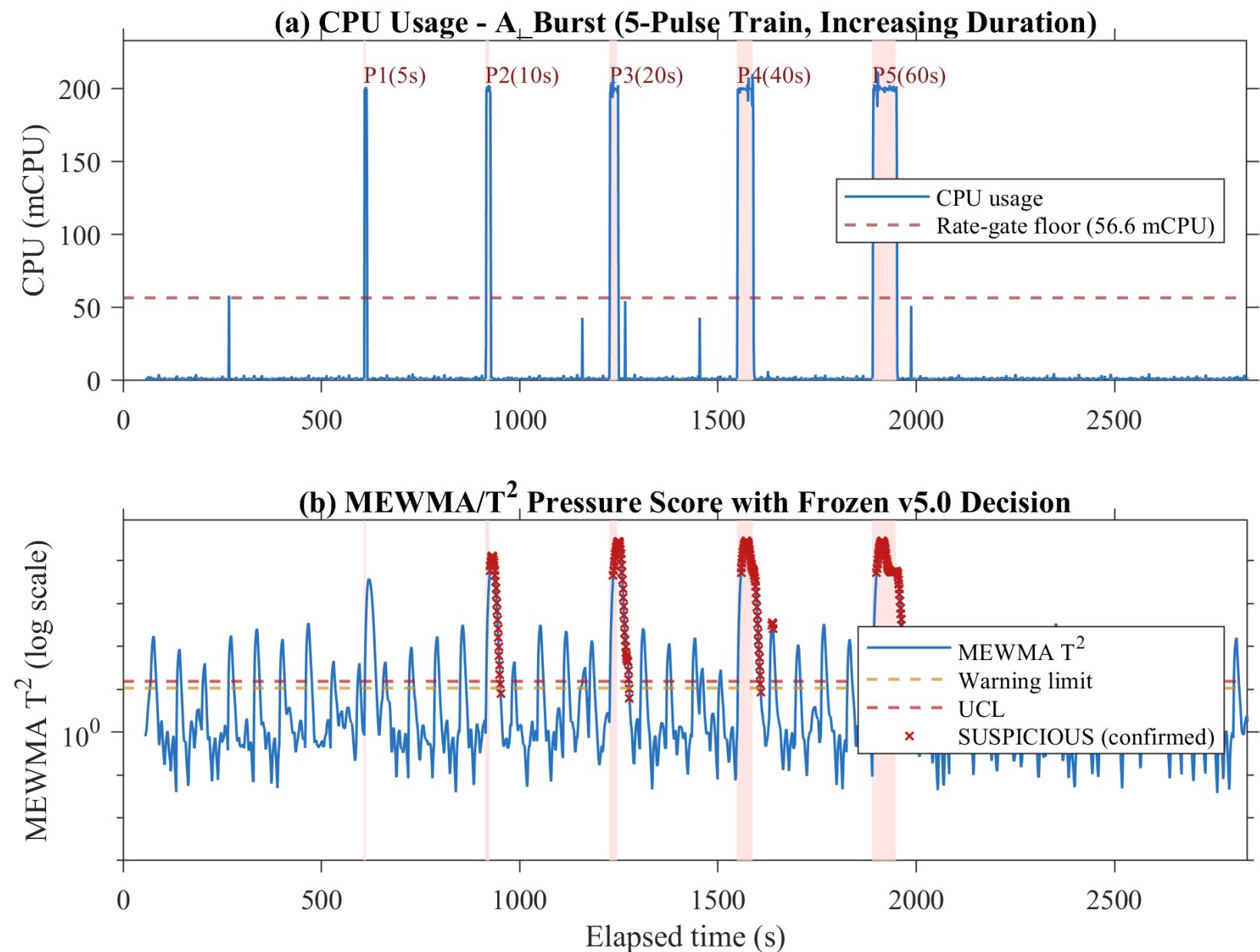
Evaluation Using R6 Dataset (CPU + Memory Combined)



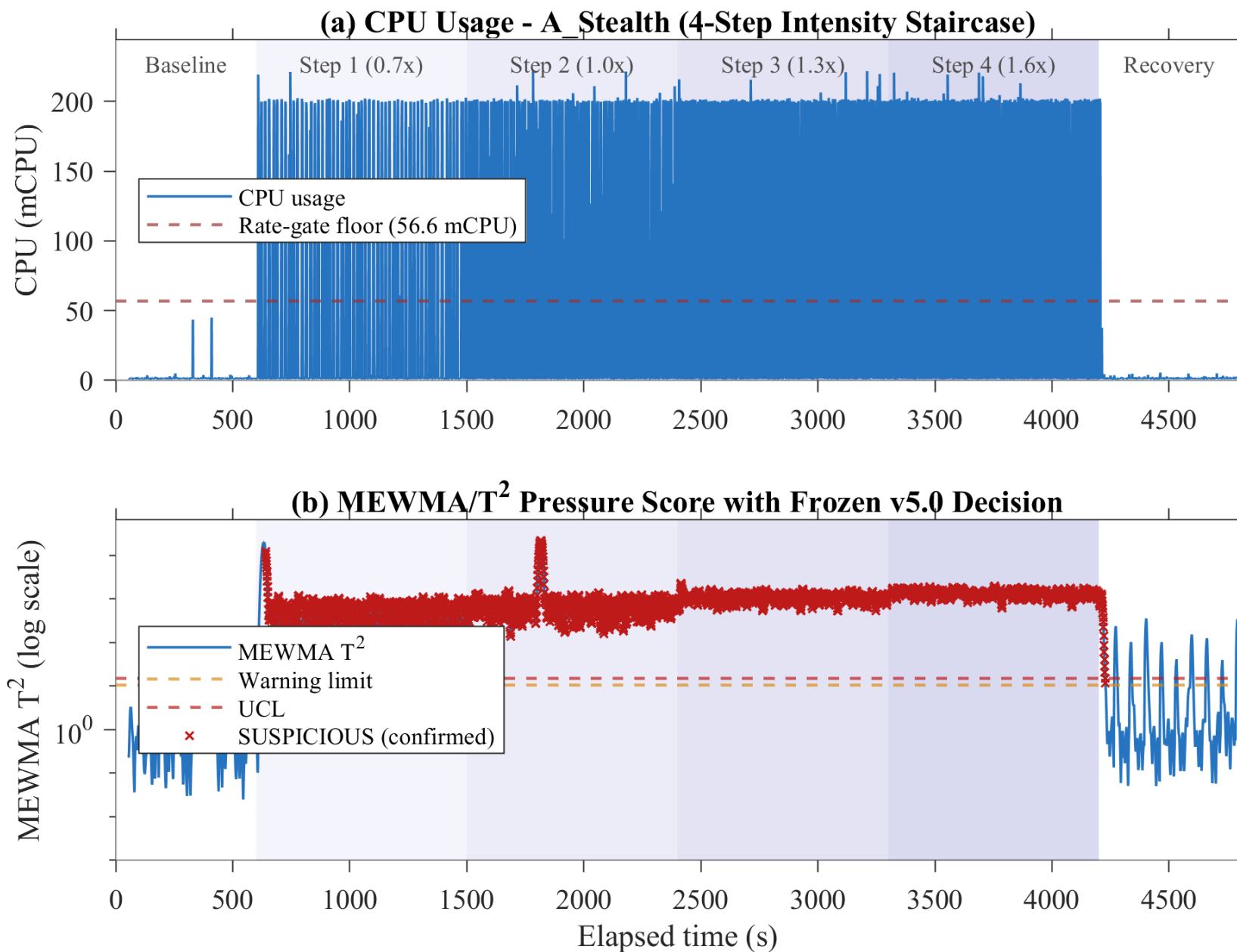
Evaluation Using A_Memory Dataset (Memory Exhaustion Attack)



Evaluation Using A_Burst Dataset (CPU Burst Attack)



Evaluation Using A_Stealth Dataset (Low and Slow Attack)



Phase Durations of the Attacks

Scenario	Baseline (Pre-attack)	Attack phase	Recovery	Total	Rationale
R1	600s	900s	300s	~30 min	Clean baseline, sustained attack, confirm de-escalation
A5	(benign throughout)	900s benign window measured	n/a	~25 min	Legitimate-load FP test
A_Stealth	600s	4x900s staircase	600s	80 min	Each intensity held long enough to reach steady state
A_Burst	600s	5 pulses (5/10/20/40/60s)+300s cooldown window for each pulse	600s	~47 min	Increasing durations to find the detection floor
A_Memory	600s	300s ramp + 1200s hold	600s	45 min	Gradual then sustained memory pressure
R6	~900s	~3600s (58 min)	~1200s	~95 min	Endurance test of a combined attack

Different durations across scenarios

A_Stealth needs each of 4 intensity levels to reach STEADY STATE (hence 900s per step)

A_Burst needs escalating durations to find the MINIMUM detectable length

R6 needs ENDURANCE. A ~600s baseline (>>20x the MEWMA warmup of 20 states) gives every scenario a stable clean reference regardless of its attack-phase length.

Confusion Matrices - Attack Scenarios

Dataset	TP	FN	FP	TN	TPR	Note
R1 (attack window)	856	44	0	544	95.11%	Recovery decay: 19 samples (<=18s tail)
R6	3464	18	0	835	99.48%	Recovery decay: 46 samples (3.85%)
A_Memory	1482	18	0	539	98.80%	Recovery decay: 30 samples (5.0% tail)
A_Stealth	3560	40	0	544	98.89%	All FN in Step1 (0.7x floor onset)

A_Burst per-pulse detail:

Pulse	Duration	Detected?	Latency	Verdict
1	5s	No	-	Correctly below 9s theoretical floor (FN by design)
2	10s	Yes	11.0s	TP
3	20s	Yes	10.0s	TP
4	40s	Yes	10.0s	TP
5	60s	Yes	11.0s	TP

Attack-duration validation against the security literature

Our scenario	Duration pattern	Literature convention	Reference
R1, R6 (sustained)	900s-3600s attack phase	Sustained resource-exhaustion studies typically run >=10 min to reach steady-state degradation	MITRE ATT&CK T1499 (Endpoint DoS) — general technique taxonomy for resource exhaustion
A_Burst (pulsed)	5-60s on-periods	Sub-minute pulsing/low-rate DoS (shrew attacks) collapse throughput with short bursts	Kuzmanovic & Knightly, "Low-Rate TCP-Targeted DoS Attacks", ACM SIGCOMM 2003
A_Stealth (low-rate)	Near-floor intensity, sustained	Low-rate attacks aimed at degrading QoS rather than full denial	Guirguis, Bestavros, Matta, "Reduction of Quality (RoQ) Attacks", IEEE INFOCOM 2005
General incident timelines	Baseline >> warmup	Standard incident-handling guidance on detection/response timing	NIST SP 800-61 Rev.2, Computer Security Incident Handling Guide

[1]. O. Ledoit and M. Wolf, “A well-conditioned estimator for large-dimensional covariance matrices,” *Journal of Multivariate Analysis*, vol. 88, no. 2, pp. 365–411, Jul. 2003, doi: 10.1016/s0047-259x(03)00096-4.

https://en.wikipedia.org/wiki/Hotelling%27s_T-squared_distribution

[2] Hotellings_Two-Sample_T2.pdf